



14.5. Merge Sort

Previous Section:

 [14.4. Counting Sort](#)

Contents:

[1. The Merge Sort Algorithm](#)

[2. Demonstration of Merge Sort](#)

[3. Implementation of Merge Sort](#)

[Summary](#)

[References](#)

Merge Sort is a **Divide-and-Conquer** sorting algorithm that repeatedly divides an unsorted sequence into smaller sub-sequences until each sub-sequence contains only one element.

Since a single element is already sorted, the algorithm then merges these sub-sequences back together while comparing their elements, producing a fully sorted sequence.

1. The Merge Sort Algorithm

The Merge Sort algorithm can be described as follows:

Given a sequence **C**, we divide it into two halves, **A** and **B**. After recursively sorting both halves, we merge them into a new sorted sequence. During the merge process, two pointers are used:

- Pointer **i** starts at the beginning of sub-sequence **A**.
- Pointer **j** starts at the beginning of sub-sequence **B**.

The algorithm repeatedly compares `A[i]` and `B[j]`:

- If `A[i]` is smaller, place it into the merged sequence and increment `i`.
- Otherwise, place `B[j]` into the merged sequence and increment `j`.

When one sub-sequence has been completely processed, all remaining elements from the other sub-sequence are copied into the merged sequence.

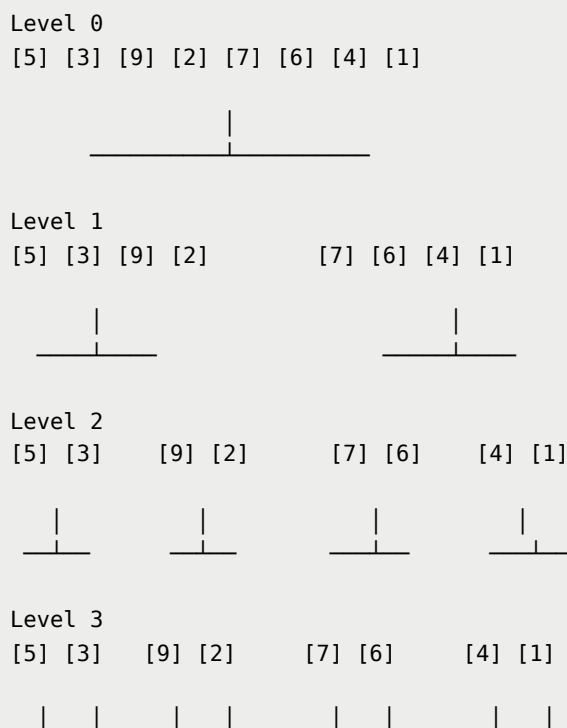
Merge Sort has a **time complexity of $O(n \log n)$** in the best, average, and worst cases, making it significantly faster than Bubble Sort, Selection Sort, and Insertion Sort for large datasets. However, this performance comes at the cost of requiring $O(n)$ additional auxiliary memory for the merging process.

2. Demonstration of Merge Sort

Consider the following array:

```
[5] [3] [9] [2] [7] [6] [4] [1]
```

- Stage 1 – Divide the array: The array is repeatedly divided into two halves until every sub-array contains only one element.



```
Level 4
[5] [3] [9] [2] [7] [6] [4] [1]
```

At this point, every sub-array contains exactly one element, so each is already considered sorted.

- Stage 2 – Merge the sub-arrays: Now the algorithm merges neighboring sub-arrays while maintaining sorted order.

```
Single elements
[5] [3] [9] [2] [7] [6] [4] [1]
    |   |   |   |
Merge into pairs
[3 5] [2 9] [6 7] [1 4]
      |       |
Merge into groups of four
[2 3 5 9] [1 4 6 7]
          |
Final merge
[1 2 3 4 5 6 7 9]
```

- **Example of one merge operation**

Suppose we merge the following two sorted arrays:

```
A = [2 5 9]
B = [1 4 7]
```

Initially, the arrays look like this:

```
A : [2] [5] [9]
    ↑i
```

```
B : [1] [4] [7]
      ↑j

Result: []
```

Compare **2** and **1** → Result: **[1]** . Move pointer **j** .

```
A : [2] [5] [9]
      ↑i

B : [1] [4] [7]
      ↑j
```

Compare **2** and **4** → Result: **[1 2]** . Move pointer **i** .

```
A : [2] [5] [9]
      ↑i

B : [1] [4] [7]
      ↑j
```

Compare **5** and **4** → Result: **[1 2 4]** . Move pointer **j** .

```
A : [2] [5] [9]
      ↑i

B : [1] [4] [7]
      ↑j
```

Compare **5** and **7** → Result: **[1 2 4 5]** . Move pointer **i** .

```
A : [2] [5] [9]
      ↑i

B : [1] [4] [7]
      ↑j
```

Compare **9** and **7** → Result: **[1 2 4 5 7]** . Pointer **j** reaches the end of array **B**, so copy the remaining elements from **A**.

```
Result: [1 2 4 5 7 9]
```

The two sorted arrays have now been successfully merged into one larger sorted array.

3. Implementation of Merge Sort

Merge Sort is naturally implemented using **recursion**.

```
def merge(left, right): # Function to merge the array
    a = []
    n1 = len(left)
    n2 = len(right)
    i, j, k = 0, 0, 0
    while (i < n1) and (j < n2):
        if left[i] < right[j]:
            a.append(left[i])
            i += 1
            k += 1
        elif right[j] < left[i]:
            a.append(right[j])
            j += 1
            k += 1
        else:
            a.append(left[i])
            i += 1
            j += 1
            k += 1
    while i < n1:
        a.append(left[i])
        i += 1
        k += 1
    while j < n2:
        a.append(right[j])
        j += 1
        k += 1
    return a
```

```
def merge_sort(a): # [Time(n), Space(n) = O(n log n), 0
(n)]
    if len(a) == 1 or len(a) == 0: # If array has only one
value or is empty
        return a
    else:
        low = 0
        high = len(a) - 1
        mid = (low + high) // 2
        left = merge_sort(a[:mid + 1])
        right = merge_sort(a[mid + 1:])
        arr = merge(left, right)
        return arr
```

The algorithm repeatedly divides the array into two smaller halves until each sub-array contains only one element, which serves as the **base case** because a single element is already sorted.

During the recursive calls, the left and right halves are sorted independently. As the recursion returns, the sorted halves are merged into larger sorted arrays until the entire original array becomes sorted.

There are two common implementation approaches:

- **Single auxiliary array:** Uses one temporary array together with indices (or pointers) to merge the two sorted halves back into the original array.
- **Multiple temporary arrays:** Creates separate temporary arrays for the left and right halves before merging. This approach is often easier to understand and may simplify the implementation, although it requires additional memory allocations.

Both approaches produce the same **$O(n \log n)$** time complexity, while requiring **$O(n)$** auxiliary space for the merging process.

Summary

Merge Sort is a Divide-and-Conquer sorting algorithm that recursively splits an array into smaller sub-arrays until each contains only one element. It then

merges these sub-arrays in sorted order to produce the final sorted sequence.

Because each level of recursion processes all elements once and there are $\log_2(n)$ levels, Merge Sort achieves a time complexity of $O(n \log n)$ regardless of the initial order of the data.

Although it requires $O(n)$ additional memory for merging, its predictable performance and efficiency make it one of the most widely used sorting algorithms for large datasets.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqqBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/merge-sort/>

<https://www.programiz.com/dsa/merge-sort>

https://www.youtube.com/watch?v=mB5HXBb_HY8

Next Section:

 [14.6. Quick Sort](#)